

Professional Web-Based Text Editor: Comprehensive Research and Development Plan

The development of a production-grade, web-based rich-text editor capable of rivaling desktop industry standards like Microsoft Word and Google Docs remains one of the most demanding engineering challenges in modern application development. In highly regulated and complex environments—such as the enterprise document management systems critical to the energy sector, which demand strict compliance, audit readiness, and precise formatting control—traditional approaches to browser-based editing fall short¹. For decades, the industry relied heavily on the DOM's contenteditable attribute, wrestling with unstructured HTML to represent highly structured, paginated documents³.

However, the paradigm has shifted. The introduction of advanced web standards—including the HTML-in-Canvas API, complex state-driven editing frameworks, and Conflict-free Replicated Data Types (CRDTs)—has fundamentally altered how developers approach document rendering, text layout, and real-time collaboration⁵. This comprehensive research and development plan outlines the architecture, user experience design, detailed feature implementation, and infrastructure required to build a high-fidelity, paginated, collaborative word processor in the browser.

UI and UX Research and Layout Architecture

To compete with established enterprise word processors, the application must balance immense functional density with cognitive clarity. The layout must accommodate desktop power users while maintaining responsiveness for tablet and mobile interfaces.

Interface Paradigms

The user interface must be context-aware and highly organized, bridging the gap between deep feature sets and immediate accessibility.

- **The Ribbon Toolbar:** For desktop environments, the traditional Ribbon interface remains superior for discoverability. It categorizes hundreds of features into distinct semantic tabs (e.g., Home, Insert, Layout, References, Review, View). A modern implementation utilizes a collapsible CSS Grid or Flexbox architecture, allowing the Ribbon to seamlessly condense into a single-line toolbar for intermediate users or smaller screens.
- **Contextual and Floating Toolbars:** When selecting an object, such as a floating image, table, or custom shape, a floating toolbar should immediately present object-specific controls (e.g., wrap text, borders, vertical alignment, crop) near the cursor. This reduces mouse travel distance and prevents the user from hunting through the main Ribbon.
- **Slash-Commands:** Integrating a / command menu provides rapid, keyboard-centric content insertion. By typing /table or /heading 2, power users can invoke complex schema structures without ever leaving the document canvas or reaching for the mouse.

- ## The Document Canvas, Zoom, and Ruler Mechanics

Adjacent to the canvas, a horizontal and vertical ruler provides precise spatial control. The horizontal ruler contains draggable sliders for paragraph layout¹¹. These UI elements must bind directly to the internal data model, updating the indent attributes of the currently selected paragraph node in real-time¹³.

```

+-----+
| Global Header: [Logo] [File Menu] [Undo/Redo] [Document Name] [Collab Avatars] |
+-----+
| Ribbon Toolbar (Collapsible) || [Home] [Insert] [Layout] [References] [Review] [View] || [Font:
Arial v] [Size: 11 v] [B I U S] | [Format Painter] | [Align...] |
+-----+
| | Ruler (Horizontal) [First Line].....[Right Indent] | | R
+-----+ S | | U | | | | L | [Page 1
Canvas] | D | | E | +-----+ | E | | R | | | | B | |
| Document content rendered here... | | A | | (V) | | Images, tables, text blocks... | | R | | | | | | |
+-----+ | |
+-----+
| Status Bar: [Page 1 of 10] [Word Count: 1,024] [Spell Check] [- Zoom +] |
+-----+

```

The fundamental decision in building a text editor is selecting the rendering engine and state

management framework.

The DOM vs. Canvas Debate

Historically, web applications utilized the DOM via contenteditable. The DOM natively provides text layout, accessibility, right-click context menus, and internationalization (IME) out of the box⁵. However, paginating continuous text, placing absolute objects across page boundaries, and scaling performance for documents exceeding one hundred pages breaks down in the DOM due to severe reflow bottlenecks and memory limitations¹⁴.

Major industry players abandoned pure DOM-based rendering in 2021, migrating to custom HTML5 Canvas engines to gain pixel-perfect control over cross-platform layout, identical print outputs, and high-performance scrolling¹⁷. Recently, browser vendors introduced an experimental HTML-in-Canvas API, bridging this gap by allowing developers to draw live, interactable DOM nodes directly into a WebGL or Canvas texture, preserving accessibility and DOM features while leveraging low-level GPU graphics performance⁵.

Technical Framework Comparison

Framework	Architecture Type	Pros	Cons	Recommendation Status
ProseMirror	Schema-driven JSON state, headless	Strict schema enforcement, industry-standard CRDT collaboration, extremely extensible ¹⁹ .	Steep learning curve, requires building UI and layout from scratch.	Highly Recommended
Lexical	React-first, double-buffering	Excellent React integration, accessibility built-in, modular ²¹ .	Performance degrades heavily with massive node counts; GC issues on large documents ²³ .	Not Recommended for massive enterprise docs
Slate.js	React-driven JSON tree	Highly customizable, native React rendering.	Weak built-in collaboration, historically unstable API, no native format	Viable alternative, but inferior collab to ProseMirror

			painter ²⁵ .	
TipTap	ProseMirror wrapper	Excellent developer ergonomics, rapid deployment ³ .	Abstracts low-level ProseMirror APIs, limiting extreme edge-case customization ³ .	Recommended for MVP, migrate to raw PM later
CKEditor 5	Custom data model	Feature-rich out of the box, strong PDF/Word export ²⁶ .	Opinionated architecture makes building highly custom canvas pagination difficult ²⁵ .	Not Recommended for custom canvas
Quill / Editor.js	Delta / Block-based	Simple API, great for basic blogs and comments.	Cannot support complex nested structures, multi-page layouts, or robust tracked changes.	Rejected

Architecture Decision and Document Data Model

The architecture will utilize a **Hybrid Frontend Approach**: React for the application shell (Ribbon, Sidebars, Menus) and **ProseMirror** as the core document state engine, interfacing with a custom paginated Canvas/DOM view layer²⁸. ProseMirror is selected because its strict schema allows the definition of rigid enterprise rules (e.g., an ordered list item must contain a paragraph block) and its transaction-based update cycle is perfectly suited for complex real-time collaboration²⁰.

ProseMirror represents documents not as HTML strings, but as a hierarchical JSON tree of nodes and marks³¹.

- **Nodes:** Structural elements (paragraphs, headings, images, lists, tables).
- **Marks:** Formatting applied to inline text (bold, italic, links, font-family).

A minimal JSON representation of this data model illustrates how formatting is strictly bound to text nodes rather than arbitrary HTML tags:

```

JSON
{
  "type": "doc",
  "content": [
    {
      "type": "paragraph",
      "attrs": { "textAlign": "left", "indent": 0 },
      "content": [
        {
          "type": "text", "text": "This represents "
        },
        {
          "type": "text", "marks": [
            { "type": "fontFamily", "attrs": { "font": "Arial" }, "type": "bold", "text":
"structured"
          },
          {
            "type": "text", "text": " data."
          }
        ]
      ]
    }
  ]
}

```

Core Editing and Formatting Engine

Building the editing tools requires intercepting native browser behaviors and translating them into deterministic state transactions.

Clipboard Operations and Format Painting

Implementing a robust clipboard requires bypassing the browser's default contenteditable paste behaviors, which inject messy inline CSS and malicious tags. The application will utilize the Async Clipboard API (`navigator.clipboard`).

Clipboard Permissions and Fallbacks: The Async Clipboard API requires secure contexts (HTTPS) and explicit user permissions to read clipboard data³⁴. It supports reading and writing multiple MIME types simultaneously (e.g., `text/plain`, `text/html`, `image/png`)³⁵. To copy highly complex internal schemas without losing fidelity, the application will use "Web Custom Formats" by prepending the MIME type with `web` (e.g., `web application/x-editor-state`)³⁷. If a user pastes from Microsoft Word, the API reads the `text/html` payload.

- *Security and Sanitization:* This HTML payload must be heavily sanitized to prevent Cross-Site Scripting (XSS). ProseMirror processes this through strict parseDOM rules, stripping out unrecognized elements, stripping `<script>` tags, and mapping valid HTML into the safe internal JSON schema³¹.

Format Painter Implementation: The Format Painter replicates complex typographical styles

across the document. When the tool is activated, the application executes a method to verify if the currently selected text has uniform formatting (e.g., ensuring the selection isn't half Arial and half Times New Roman)³⁸. The active marks (font family, size, color) are captured and stored in the application state. The cursor updates to a paintbrush icon. Upon the user's next text selection, a transaction is dispatched that iterates over the new NodeRange, aggressively removing incompatible marks and applying the stored marks²⁶. Double-clicking the Format Painter invokes a "sticky mode," allowing the user to paint multiple non-contiguous sections until the ESC key is pressed⁴⁰.

Font Tools and Formatting Preservation

Font tools govern typography: font family, size, case, bold, italic, underline, strikethrough, subscript, superscript, text effects, highlight, and color. In the schema, these are defined as marks. Because marks are independent data objects, they seamlessly overlap without causing the catastrophic nested tag issues common in raw HTML (e.g., `<b><i><u>Text</u></i></b>`).

- *Clear Formatting*: This tool dispatches a transaction that iterates through the selected text range, explicitly stripping all marks from the text nodes and resetting paragraph attributes to their default state.
- *Formatting Preservation*: When splitting a paragraph (pressing Enter), ProseMirror's default behavior carries the storedMarks from the end of the previous line into the new line, ensuring continuity of formatting⁴¹.

Paragraph Tools, Structured Lists, and Styling

Paragraph tools govern block-level structure: bulleted, numbered, and multilevel lists, sorting, alignment, spacing, shading, and borders.

List Nesting and Mechanics: Lists are notoriously fragile in web editors. Standard HTML allows lists to be malformed. The application will utilize a strict schema, likely adapted from prosemirror-schema-list²⁹. A `bullet_list` node can only contain `list_item` nodes. To prevent broken nesting, a `list_item` strictly expects block content (like a paragraph) before allowing a nested `bullet_list`³⁰. Sinking (indenting) a list item involves a complex tree transformation that wraps the target item inside a new sub-list appended to the preceding list item, ensuring the logical hierarchy is never corrupted²⁹.

Formatting Marks (Non-Printing Characters): Professional editors allow users to toggle formatting marks (e.g., paragraph returns ¶, spaces, tabs)⁴⁶. This is implemented via a ProseMirror plugin that generates non-interactive `Decoration.inline` and `Decoration.widget` elements in the view layer. These spans inject visual symbols via `CSS ::after` pseudoclasses, rendering them visible without modifying the actual document text data or affecting print exports⁴⁷.

Styles System and Themes: A robust Styles system (Normal, Heading 1, Quote, Code Block) enables document-wide consistency. Styles are defined as specific node types or node attributes in the schema. When a user updates a Style definition (e.g., changing Heading 1 from blue to black), the application queries the entire document tree for that node type and dispatches a single transaction updating their attributes. Themes operate a layer above,

defining global CSS variable palettes (e.g., `--primary-color`) that the editor respects, allowing instant document-wide aesthetic overhauls.

Editing Tools: Advanced Search Mechanics

Searching and replacing across massive documents using native DOM APIs is inherently slow and breaks easily across DOM node boundaries.

- **Search Engine:** The application implements search at the data-model level. A custom indexing algorithm processes the query (literal or Regular Expression) against the flattened text representation of the document⁴⁹.
- **Highlighting and Replacement:** Matches return start and end coordinates. The editor dispatches a transaction applying `Decoration.inline` to highlight these ranges⁴⁸. Replacing text utilizes a `ReplaceWith` transaction; crucially, the system maps the document positions as the text shifts, automatically adjusting subsequent match coordinates so the index remains accurate⁵⁰.
- **Select Similar Formatting:** This advanced feature is implemented by analyzing the marks on the current selection, executing a document-wide traversal, and creating a non-contiguous `NodeSelection` of all text blocks sharing identical marks.

Page Setup, Canvas Layout, and Object Controls

The distinguishing factor between a simple rich-text field and a professional word processor is paginated editing. Browsers natively render continuous, unpaginated scrolls. Imposing physical paper constraints requires a highly specialized layout engine.

Paginated Editing and Rendering Pipeline

Relying on DOM nodes and CSS page breaks (`break-inside: avoid`) is insufficient for real-time interactive editing, as it fails to properly split complex tables or paragraphs across pages predictably⁵¹.

The Solution: The editor retains the document as one continuous JSON data structure. However, the view layer implements a custom layout and pagination pipeline. A Web Worker calculates character widths, hyphenation points (via dictionary lookups), line heights, and margins to determine exact break points⁵¹. The renderer then paints the document visually onto multiple bounded DOM containers or Canvas layers representing distinct physical pages²⁸.

- **Page Setup:** Attributes such as margins, orientation, paper size, and columns are stored in the root doc node attributes. Altering these attributes triggers the layout engine to recalculate and reflow the entire document instantly.

Paragraph Layout: Indents and Spacing

The horizontal ruler is intrinsically linked to paragraph layout.

- **Left and Right Indents:** These shift the entire bounding box of the paragraph, controlled by square markers on the ruler¹¹.
- **First-Line and Hanging Indents:** A first-line indent applies an offset to the first text line, controlled by the top-left triangular slider¹¹. A hanging indent leaves the first line flush while indenting subsequent lines, controlled by the bottom-left slider¹¹. These values are

stored as attributes (e.g., `firstLineIndent: 0.5`) on the paragraph node and translated into precise layout coordinates by the rendering engine.

Object Arrangement and Layer Systems

Professional documents feature floating images, shapes, and text boxes.

- **Object Layer Systems:** Floating objects are represented as discrete nodes in the schema but rendered with absolute positioning based on page coordinates⁵⁵. A dedicated Selection Pane UI allows users to manage the z-index arrays, enabling commands like "Bring to Front" or "Send Backward."
- **Wrap Text (CSS shape-outside):** Integrating text wrapping around floating objects is complex. If using a DOM-based view, applying `float: left` and utilizing the CSS `shape-outside: polygon(...)` or `shape-outside: margin-box` property forces adjacent inline text nodes to reflow dynamically around the contours of the image⁵⁷. If utilizing a strict Canvas approach, the custom layout engine must mathematically compute the intersecting bounding boxes of the image and the text line geometries to calculate text wrapping offsets.
- **Transformations:** Grouping, rotating, and flipping objects involves wrapping multiple object nodes into a singular group node in the schema, applying a rotation matrix attribute (e.g., `transform: rotate(45deg)`), and recalculating their hit-boxes.

Advanced Features and Extensibility

Collaboration, State Management, and Undo/Redo

The application will provide seamless real-time collaboration using **Yjs**, an industry-leading Conflict-free Replicated Data Type (CRDT) library⁷.

- **Collaboration Architecture:** The ProseMirror state is directly bound to a Yjs shared document (`y-prosemirror`). As users type, actions generate transactions that Yjs transforms into highly compressed binary update patches, broadcasted to other clients via WebSockets⁶².
- **Cloud Sync and Offline Mode:** By utilizing WebRTC for peer-to-peer syncing and IndexedDB for local caching, the editor provides robust offline mode support. Upon reconnecting, Yjs deterministically resolves all offline edits without merge conflicts⁷.
- **Undo/Redo Complexity:** Standard linear undo stacks break in multiplayer environments (e.g., undoing a remote user's insertion). Yjs provides a scoped UndoManager that tracks the origin of changes. By assigning a unique client ID to local transactions and passing it into the trackedOrigins set, the undo stack strictly reverts local actions while intelligently preserving remote updates⁶³.

Track Changes, Comments, and Version History

Implementing a rigorous "Reviewing" mode involves advanced schema tracking.

- **Track Changes Data Model:** The ProseMirror schema is extended with custom marks: insertion, deletion, and modification⁶⁶. When a user deletes text in this mode, it is not removed from the state; instead, it is wrapped in a deletion mark and rendered with a

strikethrough and a specific author color⁶⁷.

- **ChangeSets:** To efficiently display history, the prosemirror-changeset library computes the differential between document states, generating spans of additions and deletions for a unified review interface⁶⁹.
- **Resolution:** Commands like applySuggestion (which permanently removes the deletion mark or finalizes the insertion) and revertSuggestion (which restores the text) manage the resolution of these tracks⁶⁶.

Spell Check, AI Assistants, and Voice Typing

- **Spell Check Overlays:** Relying on the browser's native spellcheck="true" within highly dynamic, React-rendered or Canvas-rendered environments is deeply flawed, as it constantly resets during DOM diffing and struggles with absolutely positioned nodes⁷⁰. The solution disables native spellchecking and utilizes a background Web Worker running a linguistic engine (e.g., LanguageTool API)⁷³. The worker analyzes text chunks, returns error ranges, and the main thread uses Decoration.inline to render the red squiggly underlines⁴⁸.
- **AI and Voice Integration:** AI assistants interact by analyzing document slices and proposing revisions via the Track Changes API. Voice typing integrates the Web Speech API, intercepting dictated transcripts and translating them into continuous insertion transactions.

Security, Reliability, and Export Infrastructure

Security and Performance Strategies

- **Sanitization and Malicious Imports:** Web-based editors are vulnerable to Cross-Site Scripting (XSS) via copy-pasting malicious HTML. ProseMirror's parseDOM safely strips script, iframe, and unrecognized onEvent attributes before they ever reach the document state.
- **Encryption and Permissions:** For highly sensitive documents, payload data broadcasted via WebSockets can be End-to-End Encrypted (E2EE) before transmission. Document permissions (View, Comment, Edit) are enforced at the WebSocket server level, validating JWT tokens before accepting Yjs updates⁷⁴.
- **Upload Validation:** Images dragged into the canvas are intercepted. A placeholder node with a loading spinner is inserted into the document. The binary file undergoes MIME type validation and malware scanning before being uploaded to a secure S3 bucket⁷⁶. The placeholder is then replaced with the secure CDN URL.
- **Large Document Performance:** To maintain 60 FPS on massive documents, the application employs DOM virtualization. Only the currently visible pages and a small rendering buffer are painted to the screen; off-screen pages remain purely as JSON state in memory. Multi-node formatting operations are batched into a single transaction frame to prevent layout thrashing⁷⁷.

Export and Interoperability (DOCX, PDF)

Professional interoperability demands high-fidelity export mechanisms.

- **HTML to DOCX:** The application will utilize a high-performance JavaScript library such as @turbodocx/html-to-docx to generate .docx files natively in the browser⁷⁸. By translating the internal JSON schema into clean HTML and passing it to the library alongside page setup parameters (margins, paper size), the system generates Microsoft Word-compatible files entirely client-side, reducing server load⁷⁸.
- **PDF Generation:** Because the internal pagination engine guarantees that the on-screen DOM or Canvas matches the exact print layout, PDF generation can be executed directly via browser printing APIs (window.print()) or handled server-side via Puppeteer/Playwright capturing the frozen document state²⁷.

Roadmap and Deliverables

Phase	Duration	Core Focus	Deliverables
Phase 1: MVP	Months 1-3	Architecture & Core Mechanics	Set up ProseMirror with block/inline schema. React-based ribbon toolbar. Basic font/paragraph formatting. Local IndexedDB storage.
Phase 2: Layout	Months 4-6	Pagination & Object Control	Web Worker pagination layout engine. Ruler UI with indent sliders. Object floating, alignment, and shape-outside text wrapping ⁵⁸ .
Phase 3: Multiplayer	Months 7-9	Collaboration & Track Changes	Integrate Yjs over WebSockets. Scoped local Undo/Redo ⁶⁴ . Implement track changes, comments, and review sidebar ⁶⁶ .

Phase 4: Enterprise	Months 10-12	Integrations & Export	Custom Web Worker spellcheck ⁷³ . DOCX/PDF export ⁷⁸ . AI assistant API integration. E2EE encryption modules.
--------------------------------	--------------	-----------------------	---

Risk Analysis and Mitigation

Risk	Impact	Mitigation Strategy
Pagination Reflow Latency	High UI freezing on large documents during layout shifts.	Decouple layout mathematics from the main thread. Process pagination calculations in a Web Worker, utilizing DOM virtualization to only render visible pages ⁵¹ .
Collab Conflict Resolution	"Split brain" states or loss of text during offline sync.	Rely strictly on Yjs CRDTs rather than Operational Transformation (OT). Enforce a robust backup mechanism capturing incremental document snapshots ⁷ .
Pasting Malformed Data	Corrupted document model causing the editor to crash.	Utilize strict ProseMirror parseRules. Implement a fallback to text/plain extraction if the HTML schema validation entirely fails.

Final Recommendations

To successfully engineer an enterprise-grade word processor in the browser, development teams must entirely abandon reliance on native contenteditable behaviors for layout and

document structure. The browser must be treated purely as a low-level rendering target.

1. **Adopt ProseMirror:** The strict, predictable schema enforcement and powerful transaction pipeline make it the premier framework capable of handling the complexity of a Microsoft Word equivalent.
2. **Invest in a Custom Layout Pipeline:** Do not attempt to rely on CSS page breaks for pagination. True pagination requires a dedicated layout engine that calculates character metrics and mathematically determines page breaks, feeding that data into a virtualized DOM or HTML-in-Canvas view.
3. **Prioritize CRDTs from Inception:** Do not build the application as a single-player tool and attempt to bolt on collaboration later. The fundamental document structure must be built around Yjs shared types from day one to ensure data integrity, offline support, and track-changes compatibility.

By executing this architecture, the application will achieve native-like performance, pixel-perfect document fidelity, and seamless real-time collaboration, successfully delivering a next-generation web document experience capable of satisfying the most stringent enterprise requirements.

Works cited

1. Houston Data Storage Solutions, Document Scanning, and Shredding Service - Access, <https://www.accesscorp.com/location/texas/houston/>
2. Document Control & Records Manager - Blue Energy - BeBee, <https://bebee.com/us/jobs/document-control-records-manager-blue-energy-gre-ater-houston--theirstack-668004012>
3. Rich text editors and rendering engines | Hacker News, <https://news.ycombinator.com/item?id=36600434>
4. Document Object Model (DOM) - Web APIs - MDN Web Docs, https://developer.mozilla.org/en-US/docs/Web/API/Document_Object_Model
5. Introducing the HTML-in-Canvas API origin trial | Blog - Chrome for Developers, <https://developer.chrome.com/blog/html-in-canvas-origin-trial>
6. Google I/O 2026 quietly ended a 20-year-old web problem — meet the HTML-in-Canvas API, <https://dev.to/manikant92/google-io-2026-quietly-ended-a-20-year-old-web-problem-meet-the-html-in-canvas-api-4h9d>
7. yjs - NPM, <https://www.npmjs.com/package/yjs>
8. zoom CSS property - MDN Web Docs, <https://developer.mozilla.org/en-US/docs/Web/CSS/Reference/Properties/zoom>
9. Zooming on a point with CSS3 transform scale - Stack Overflow, <https://stackoverflow.com/questions/27603929/zooming-on-a-point-with-css3-transform-scale>
10. Using css transform results in incorrect behavior · Issue #324 · codemirror/dev - GitHub, <https://github.com/codemirror/dev/issues/324>
11. Using the ruler in Word | Microsoft Support, <https://support.microsoft.com/en-us/word/using-the-ruler-in-word>

12. Formatting tools: The ruler - ScrivenerVirgin,
<https://www.scrivenervirgin.com/2021/03/formatting-tools-the-ruler/>
13. Ruler indents jump all over the place & how to set margins for Word 1" margins - Scrivener,
<https://forum.literatureandlatte.com/t/ruler-indents-jump-all-over-the-place-how-to-set-margins-for-word-1-margins/145794>
14. How Browsers Render Web Pages - ByteByteGo,
<https://bytebytego.com/guides/how-does-the-browser-render-a-web-page/>
15. What is the DOM & How Does HTML Rendering Happen in Browsers? | by Muhammed Erdinç | Teknasyon Engineering | Medium,
<https://medium.com/teknasyon-engineering/what-is-the-dom-how-does-html-rendering-happen-in-browsers-cbeb12bdfea6>
16. How does a Browser render a Webpage? - DEV Community,
<https://dev.to/anuradha9712/how-does-a-browser-render-a-webpage-2en8>
17. The first document editor: a simple but risky bet - OnlyOffice,
<https://www.onlyoffice.com/blog/2026/06/the-first-doc-editor>
18. Google Docs Switches to Canvas Rendering, Sidelining the DOM - The New Stack,
<https://thenewstack.io/google-docs-switches-to-canvas-rendering-sidelining-the-dom/>
19. ProseMirror, <https://prosemirror.net/>
20. Lexical vs ProseMirror 2026: Skip Both, Save 8 Weeks + \$60K | Eddyter,
<https://eddyter.com/blogs/lexical-vs-prosemirror-2026>
21. Introduction - Lexical, <https://lexical.dev/docs/intro>
22. Lexical, an extensible text editor framework : r/javascript - Reddit,
https://www.reddit.com/r/javascript/comments/u5088q/lexical_an_extensible_text_editor_framework/
23. Rich Text Editors in Action: Stress Test On Lexical and ProseMirror,
<https://emergence-engineering.com/blog/lexical-prosemirror-comparison>
24. Bug: Extremely slow performance in big documents · Issue #7422 · facebook/lexical - GitHub, <https://github.com/facebook/lexical/issues/7422>
25. Format Painter: Under pressure - TinyMCE,
<https://www.tiny.cloud/blog/under-pressure-format-painter/>
26. Format painter | CKEditor 5 Documentation,
<https://ckeditor.com/docs/ckeditor5/latest/features/format-painter.html>
27. The best way to export your WYSIWYG editor content to PDF and Word | CKEditor,
<https://ckeditor.com/blog/The-best-way-to-export-your-WYSIWYG-editor-content-to-PDF-and-Word-file-formats/>
28. Building a Canvas-Based Editor on Top of ProseMirror's State and Plugin System,
<https://discuss.prosemirror.net/t/building-a-canvas-based-editor-on-top-of-prosemirror-s-state-and-plugin-system/8982>
29. prosemirror-schema-list/src/schema-list.ts at master - GitHub,
<https://github.com/ProseMirror/prosemirror-schema-list/blob/master/src/schema-list.ts>

30. Discussion : Replacing BulletList -> ListItem with an infinitely nestable CustomListItem,
<https://discuss.prosemirror.net/t/discussion-replacing-bulleted-list-item-with-an-infinitely-nestable-customlistitem/2263>
31. The Unreasonable Effectiveness of ProseMirror Model in Rich Text Transformation,
https://smoores.dev/post/unreasonable_effectiveness_of_prosemirror/
32. ProseMirror Guide, <https://prosemirror.net/docs/guide/>
33. Document representation in JSON? - discuss.ProseMirror,
<https://discuss.prosemirror.net/t/document-representation-in-json/238>
34. Clipboard API and events - W3C, <https://www.w3.org/TR/clipboard-apis/>
35. ClipboardItem - Web APIs | MDN,
<https://developer.mozilla.org/en-US/docs/Web/API/ClipboardItem>
36. Multi-MIME Type Copying with the Async Clipboard API - Blogccasion,
<https://blog.tomayac.com/2020/03/20/multi-mime-type-copying-with-the-async-clipboard-api/>
37. Web custom formats for the Async Clipboard API | Blog - Chrome for Developers,
<https://developer.chrome.com/blog/web-custom-formats-for-the-async-clipboard-api>
38. Format Painter in ASP.NET Core: Building Custom Text Formatting with TX Text Control,
<https://www.textcontrol.com/blog/2025/09/09/format-painter-in-asp-dotnet-core-building-custom-text-formatting-with-tx-text-control/>
39. Javascript Rich Text Editor Format Painter Example - Syncfusion Demos,
<https://ej2.syncfusion.com/demos/rich-text-editor/format-painter/>
40. Javascript (ES5) Rich Text Editor Format Painter Example - Syncfusion Demos,
<https://ej2.syncfusion.com/javascript/demos/rich-text-editor/format-painter/>
41. Selection - ProseMirror Reference manual,
<https://prosemirror.net/docs/ref/version/0.20.0.html>
42. prosemirror-schema-list/src/README.md at master - GitHub,
<https://github.com/ProseMirror/prosemirror-schema-list/blob/master/src/README.md>
43. Parsing non-standard nested (un)ordered list HTML - discuss.ProseMirror,
<https://discuss.prosemirror.net/t/parsing-non-standard-nested-un-ordered-list-html/245>
44. List-only schema - discuss.ProseMirror,
<https://discuss.prosemirror.net/t/list-only-schema/3803>
45. Ordered/Bullet Lists Text Only - discuss.ProseMirror,
<https://discuss.prosemirror.net/t/ordered-bullet-lists-text-only/840>
46. How to Show Non Printing Characters in Word: Display Hidden Marks - YouTube,
<https://www.youtube.com/watch?v=hMZUsbBb2Ho>
47. guardian/prosemirror-invisibles - GitHub,
<https://github.com/guardian/prosemirror-invisibles>
48. ProseMirror DecorationSet in React: Everything I Wish Someone Had Told Me - Medium,

- <https://medium.com/@faisalmujtaba/prosemirror-decoration-set-in-react-everything-i-wish-someone-had-told-me-6262eabae7ca>
49. Search/replace functionality for ProseMirror - GitHub,
<https://github.com/ProseMirror/prosemirror-search>
 50. How to replace currently selected content in ProseMirror - Stack Overflow,
<https://stackoverflow.com/questions/61993331/how-to-replace-currently-selected-content-in-prosemirror>
 51. What's the best way to split text into multiple pages in a browser-based editor?,
<https://community.latenode.com/t/whats-the-best-way-to-split-text-into-multiple-pages-in-a-browser-based-editor/9461>
 52. Tiptap Pagination (Problems & Solutions*) | by Romik Makavana - Medium,
<https://romik-mk.medium.com/tiptap-pagination-problems-solutions-31f1a0b51e08>
 53. Tiptap Pages vs tiptap-pagination-plus | by Romik Makavana - Medium,
<https://romik-mk.medium.com/tiptap-pages-vs-tiptap-pagination-plus-38e0c7cd046d>
 54. Microsoft Word Guide: Using the Ruler for Indents - ProofreadingPal,
<https://proofreadingpal.com/proofreading-pulse/writing-guides/microsoft-word-guide-using-the-ruler-for-indents/>
 55. ProseMirror Reference manual, <https://prosemirror.net/docs/ref/version/0.5.0.html>
 56. How to get absolute position of any node? - discuss.ProseMirror,
<https://discuss.prosemirror.net/t/how-to-get-absolute-position-of-any-node/4313>
 57. shape-outside CSS property - MDN Web Docs,
<https://developer.mozilla.org/en-US/docs/Web/CSS/Reference/Properties/shape-outside>
 58. Getting Creative With shape-outside - CSS-Tricks,
<https://css-tricks.com/getting-creative-with-shape-outside/>
 59. Wrapping Content around images using CSS Shapes | by Nicholas Evans - Medium,
<https://medium.com/@nicke920/wrapping-content-around-images-using-css-shapes-2fb17c8fde4e>
 60. Creative text flows using CSS shapes - LogRocket Blog,
<https://blog.logrocket.com/creative-text-flows-using-css-shapes/>
 61. I am building a Google Docs alternative using Cloudflare Durable Objects and CRDTs - focusing on academic version control. : r/SideProject - Reddit,
https://www.reddit.com/r/SideProject/comments/1q7fb6y/i_am_building_a_google_docs_alternative_using/
 62. I'm actually in the middle of rewriting the y-prosemirror binding with Kevin Jah... | Hacker News, <https://news.ycombinator.com/item?id=47409955>
 63. Avoiding y-prosemirror "undo" - Yjs Community,
<https://discuss.yjs.dev/t/avoiding-y-prosemirror-undo/985>
 64. Y.UndoManager - Yjs Docs, <https://docs.yjs.dev/api/undo-manager>
 65. Single UndoManager Multiple Code/RichText Editors - Yjs Community,
<https://discuss.yjs.dev/t/single-undomanager-multiple-code-richtext-editors/591>

66. handlewithcare/prosemirror-suggest-changes - GitHub,
<https://github.com/handlewithcarecollective/prosemirror-suggest-changes>
67. davefowler/prosemirror-suggestion-mode - GitHub,
<https://github.com/davefowler/prosemirror-suggestion-mode>
68. Prosemirror-changeset for tracking document changes including deletions,
<https://discuss.prosemirror.net/t/prosemirror-changeset-for-tracking-document-changes-including-deletions/2524>
69. prosemirror-changeset - GitHub,
<https://github.com/prosemirror/prosemirror-changeset>
70. We are using and customizing ProseMirror exhaustively at Missive (<https://missiv...>)
| Hacker News, <https://news.ycombinator.com/item?id=18999703>
71. inputStyle = contenteditable, we may hope for browser spell checking? -
discuss.CodeMirror,
<https://discuss.codemirror.net/t/inputstyle-contenteditable-we-may-hope-for-browser-spell-checking/608>
72. Spell Checker Problems · Issue #390 - GitHub,
<https://github.com/ProseMirror/prosemirror/issues/390>
73. ProseMirror Proofread/Spell Check Library - Show,
<https://discuss.prosemirror.net/t/prosemirror-proofread-spell-check-library/7680>
74. Generate an inverse update? - Yjs Community,
<https://discuss.yjs.dev/t/generate-an-inverse-update/2765>
75. Is there a way to reject an update for a Doc? - Yjs Community,
<https://discuss.yjs.dev/t/is-there-a-way-to-reject-an-update-for-a-doc/3805>
76. Add the rich text editor control to a model-driven app - Power Apps | Microsoft
Learn,
<https://learn.microsoft.com/en-us/power-apps/maker/model-driven-apps/rich-text-editor-control>
77. Need help to improve editor performance - discuss.ProseMirror,
<https://discuss.prosemirror.net/t/need-help-to-improve-editor-performance/8860>
78. @turbodocx/html-to-docx - npm,
<https://www.npmjs.com/package/@turbodocx/html-to-docx>
79. Pure client-side HTML ↔ DOCX conversion library for JavaScript : r/webdev -
Reddit,
https://www.reddit.com/r/webdev/comments/1m9rvho/pure_clientside_html_docx_conversion_library_for/
80. HTML to DOCX Conversion JavaScript SDK - ConvertAPI,
<https://www.convertapi.com/html-to-docx/javascript>
81. DHTMLX RichText 2.0: Full-Featured Editor with the Next-Gen Engine,,
<https://dhtmlx.com/blog/dhtmlx-rich-text-editor-2-0/>
82. CSS resize - discuss.ProseMirror, <https://discuss.prosemirror.net/t/css-resize/1286>